Labo 1: Geavanceerde computerarchitectuur

Sander De Stercke 16/02/2026

Context

In dit labo wordt met behulp van een GPU en CUDA C++, een array van onbepaalde lengte N omgedraaid. Dit is een manier om de basisprincipes van GPU-programming onder de knie te krijgen.

Methodology

De werking van de arrayflipper werkt door elke aparte threat één element voor zich te laten nemen. De aantal threats hangt dus af van de grootte van de array (N). Waarbij de thread met index 0 het einde van de ingangsarray neemt (N – 1 + 0) en die op zijn huidige index plaatst in de nieuwe array. De index van de huidige thread wordt bepaalt op basis van bloknummer, blokgrootte en index binnen deze block. Deze werking is geïllustreerd in de flipArray functie in de code.

Observation

De aanpak is correct, echter was ik initieel enkele stappen vergeten omdat mijn focus initieel lag op de kernel functie. Zoals het aanmaken van geheugen en de inhoud van de array te kopiëren van host (CPU) naar device (GPU) en omgekeerd. Deze technieken vergen nog meer oefening tegen dat ik ze volledig in de vingers heb.

Conclusion

De basisprincipes van GPU-programming zijn geëxploreerd.

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel to flip an array
__global__ void flipArray(int* A, int* B, int N) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    B[index] = A[(N - 1) - index];
}

int main() {
    const int N = 64;
    int A[N], B[N];
    int *d_A, *d_B;

    // Initialize first array
    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100 ;
    }

    for (int i; i < N; i++){
        printf("%d ", A[i]);
    }

    printf("\n");

    // Allocate device memory
    cudaMalloc((void**)&d_A, N * sizeof(int));
    cudaMalloc((void**)&d_B, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 8;
    int blocksPerGrid = ((N - 1) + threadsPerBlock) / threadsPerBlock;

    // Launch kernel (1 block of N threads)
    flipArray<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, N);

    cudaDeviceSynchronize();

    // Copy result back to host
    cudaMemcpy(B, d_B, N * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < N; i++) {
        printf("%d ", B[i]);
    }

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);

    return 0;
}
```